

Software Engineering

Introduction

Outline

- Definitions and concepts
- Process and product
- Process and product properties
- Laws
- Principles

Definitions

Definition of Software Engineering

- **Multi person** construction of **multi version** software
- Multi person:
 - Issues in communication: misunderstandings, language gaps
 - Issues in coordination
- Multi version:
 - Issue in maintenance over many years

Definition of Software

- A collection of
 - Computer programs,
 - Procedures,
 - Rules,
 - Associated documentation
 - Data
- Example:
 - Requirements document,
 - Design document
 - Project plan,
 - Test plan,
 - Test cases,
 - Build scripts,
 - Deployment scripts,
 - User manuals

Software is NOT only code

Software Types

- Embedded in non software product
 - Car, washing machine, train, plane, ..
- Stand alone
 - Office suite, social network, ..
- Embedded in business process (enterprise)
 - Information system
- Embedded in production process (enterprise, factory)
 - Production line (Industry 4.0)

Software Criticality

- Safety critical: harms people or environment
 - Self driving car
 - (usually embedded software)
- Mission critical: harms business
 - Banking, finance, retail
 - (usually embedded in enterprise)
- Other

Type of software and criticality

- Embedded software can have a direct effect on real world, and be safety critical
 - Ex. Self driving car, train/plane control system
- Stand alone software does not have (usually) a direct effect on real world, and is not safety critical
 - There are exceptions, though (ex FEM tools to design bridges, buildings)

Beware

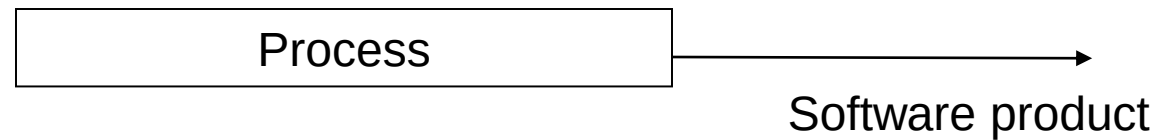
- Software is not free
- Software changes (and it is not easy to change)
- Software is not perfect
- Software is complex

Process and Product

The Software “factory”



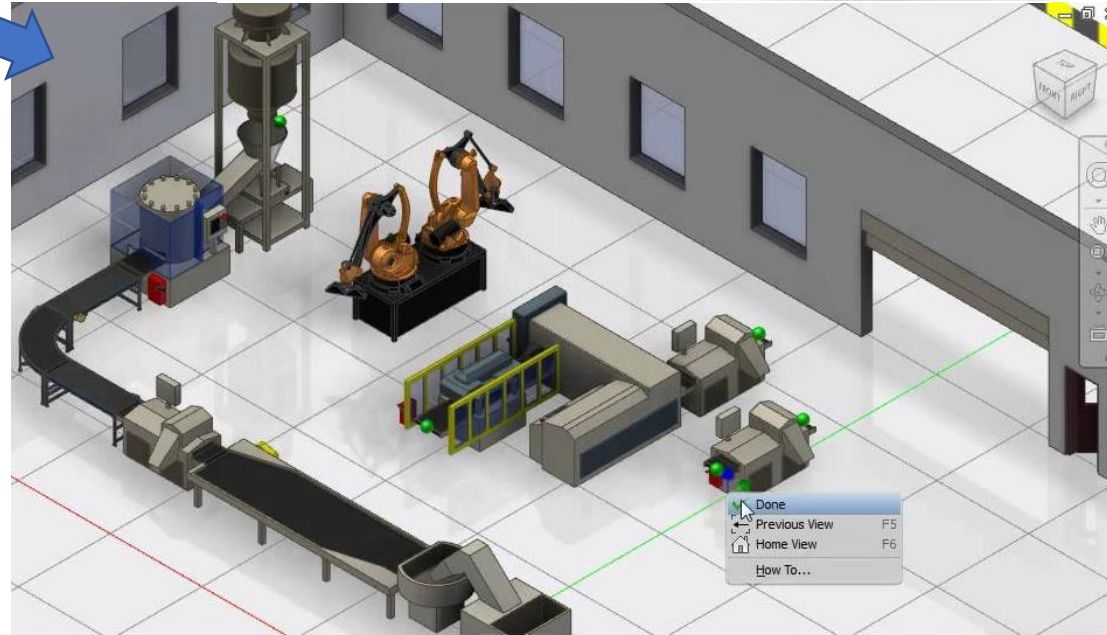
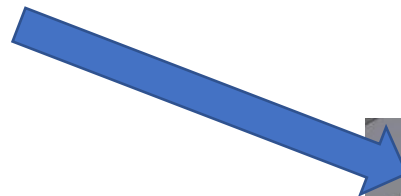
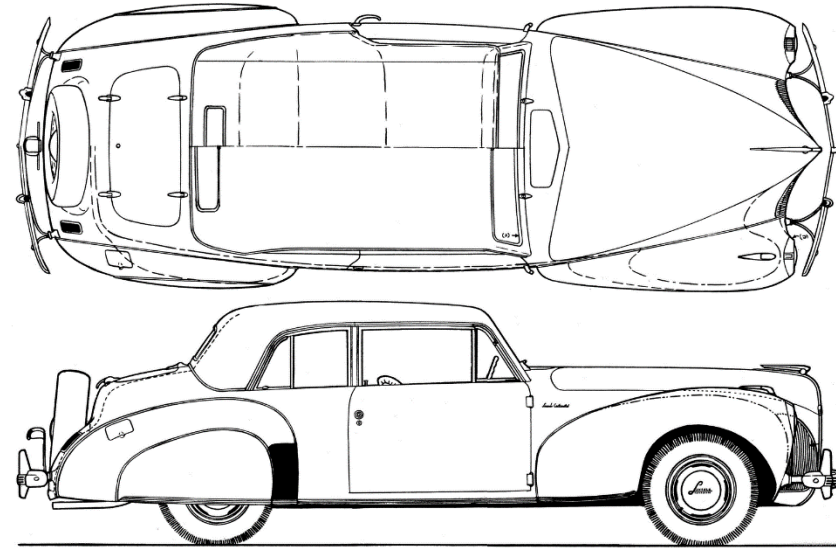
Process (the factory) and product



- Process: activities, people, tools
- Products: documents, data, code
- The quality of the product depends on the quality of the process
 - Garbage in, garbage out

Classical Engineering

- Design the product
- Design the factory
- Manufacture the product
- Maintain the product



Classical Engineering

Software Engineering

- Design the product



- Software product

- Design the factory



- Software Process

- Manufacture the product

HARDER



- Deployment and Delivery

EASIER

- Maintain the product

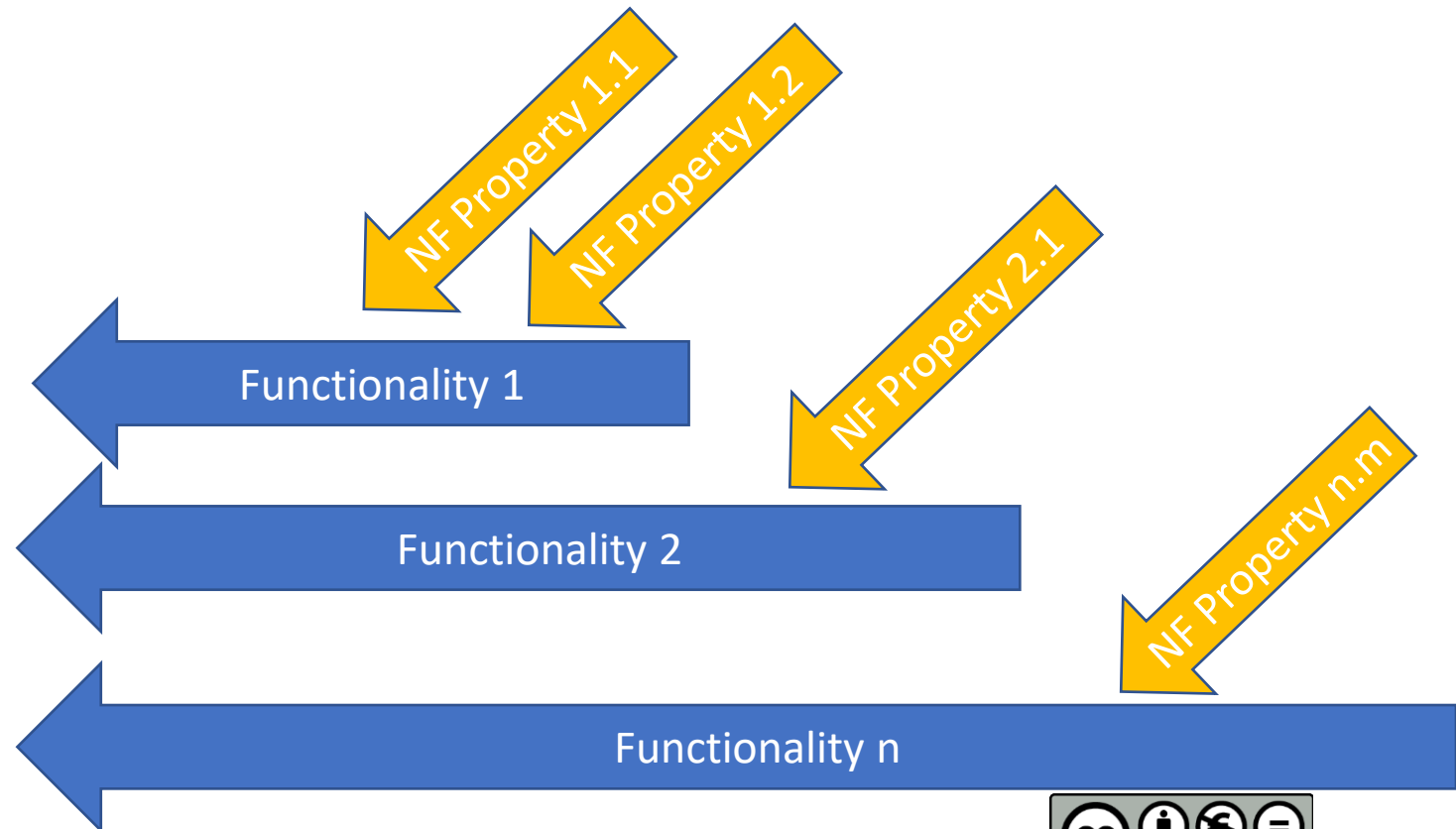


- Evolution and Maintenance

Process and Product Properties

Software product

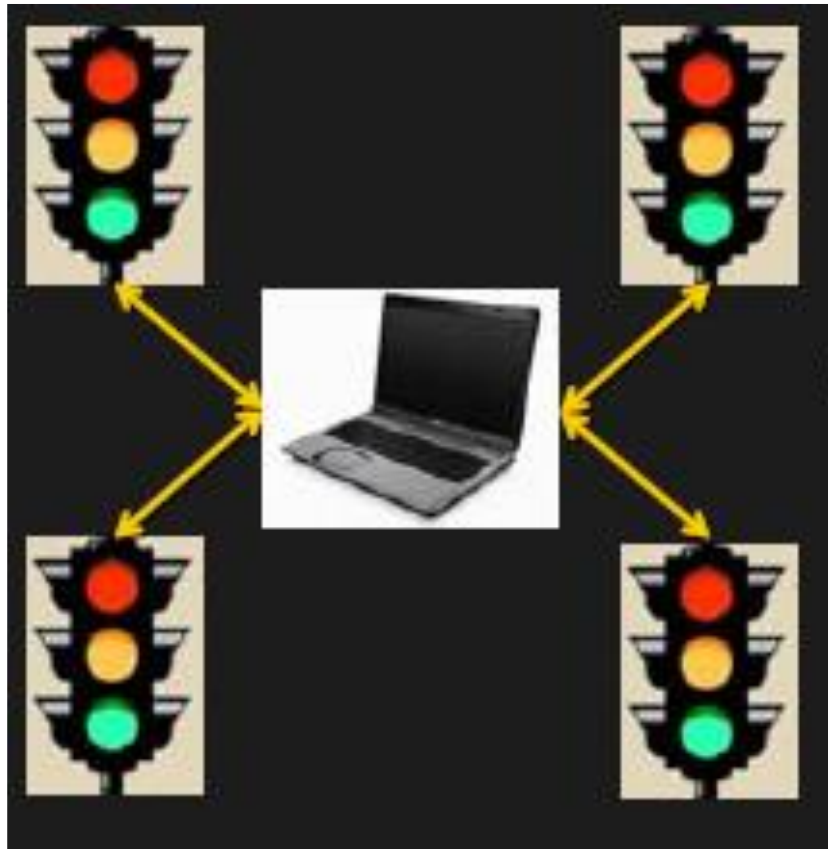
- Is characterized by
 - Functional properties
 - Non functional properties



Software Product

- Functional properties
 - Do something
 - ...

Ex traffic light controller - functionality



- Do this:
- control 4 traffic lights in a road crossing so that ..
 - ♦ Green in one direction, red in other direction during x sec
 - ♦ Flashing yellow in one direction during y sec, red in other direction
 - ♦ Red in one direction during z sec, green in other direction

Software Product

- Non functional properties
 - Correctness
 - Capability of the product to provide the intended functionality in **all** cases
 - Reliability / availability
 - Defects visible by end user per time period / Probability of defect over a time period
 - Percentage of time the product is / is not available to end user

Traffic light controller - correctness

- The intended sequence of signals is **always** satisfied
 - All possible cycles
 - All possible combinations of durations

Traffic light controller - reliability

- Reliability

- The intended sequence of signals could be not as intended
- Ex: In one cycle over 1 million

- Availability

- The controller can be down (not providing any sequence)
- Ex: downtime one hour over a one year period
- Ex availability = $(\text{one year} - \text{one hour}) / \text{one year}$

Software Product

- Non functional properties
 - Security
 - Protection from malicious access to functions / Access only to authorized users
 - Access to data
 - Safety
 - Absence of harm to persons
 - Absence of hazardous situations for persons
 - Dependability
 - Safety + security + reliability

Traffic light controller

- Safety
 - Hazardous situations:
 - Green in all directions.
 - All lights off
 - The controller is capable of avoiding them?
- Security
 - Malicious attacks to bring the controller in a hazardous situation
 - Access to private data
 - The controller / the system is capable of avoiding them ?

Software Product

- Non functional properties
 - Usability
 - Effort needed to learn using the product (installation, day to day usage)
 - Satisfaction expressed by the user
 - Existence of functions needed by the user

Usability

```
MS-DOS Prompt
Auto
C:\WINDOWS>cd ..
C:\>e:
E:\>copy c:\mymusi~1\nsync~1\*.*
c:\mymusi~1\nsync~1\N Sync & Gloria Estefan - Music Of My Heart.mp3
c:\mymusi~1\nsync~1\N Sync - Best of My Life.mp3
c:\mymusi~1\nsync~1\N Sync - Bye Bye Bye.mp3
c:\mymusi~1\nsync~1\N Sync - I Drive Myself Crazy.mp3
c:\mymusi~1\nsync~1\N Sync - I Want You Back.mp3
c:\mymusi~1\nsync~1\N Sync - Tearin' up my heart.mp3
c:\mymusi~1\nsync~1\N' Sync - God Must Have Spent A Little More Time On You.mp3
7 file(s) copied
E:\>_
```



Traffic light controller

- Usability
 - Set up of the controller, how easy to do?
 - Time to do set up for non expert user
 - (usability does not really apply to core function of controller, computing the sequence of lights)

Software Product

- Non functional properties
 - Efficiency
 - For a given function in a given context: response time
 - For a given function / for a complete product:
 - Memory
 - CPU
 - Bandwidth
 - energy used

Traffic light controller

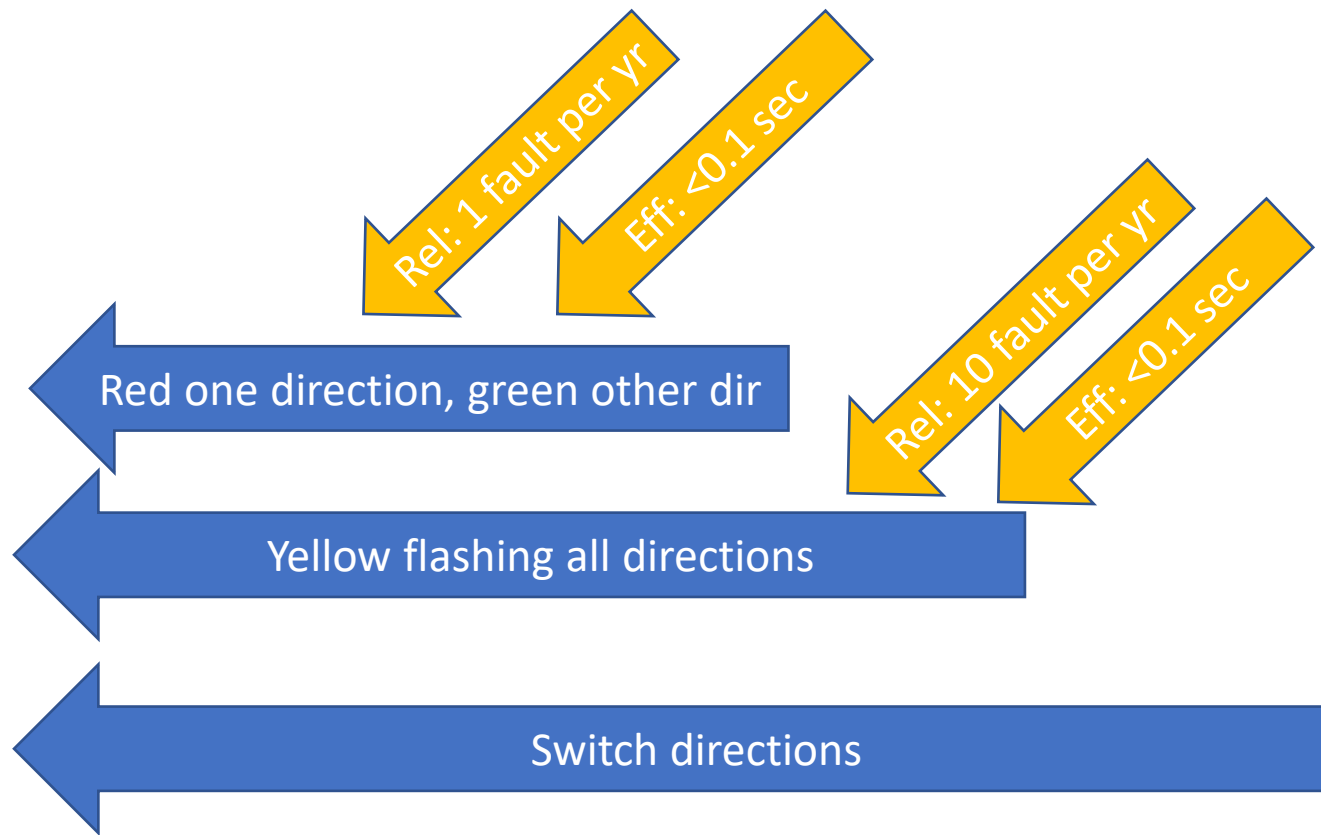
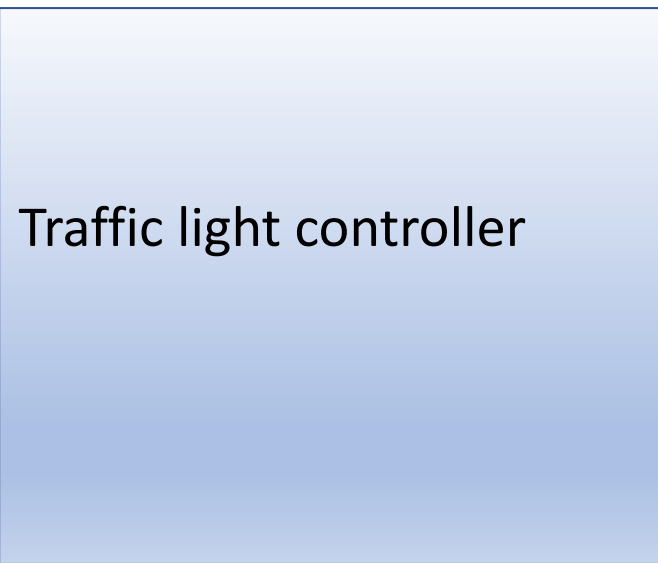
- Efficiency
 - The computation of the next light values should take < 0.1 sec
 - The controller should use less than 16Mbyte ram

Software Product

- Non functional properties
 - Maintainability
 - Effort (person hours) needed to add /modify / cancel a software function
 - Effort to fix a defect
 - Effort to deploy on a different platform (DB, OS, ..)

Traffic light controller - mantainability

- Adding a light to control left turns / right turns could require
 - Ex: one hour work?
 - Ex: one month work?
- Porting the controller from Linux to Windows could require
 - One week? One month work?
- Using mysql instead of sqllite could require
 - One week? One month work?

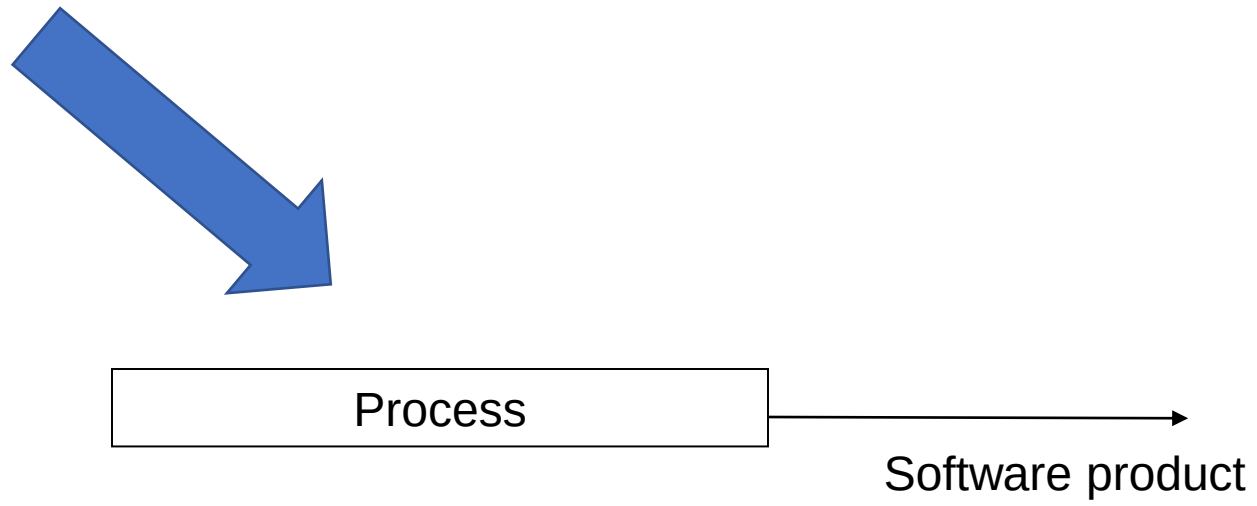


Software Product

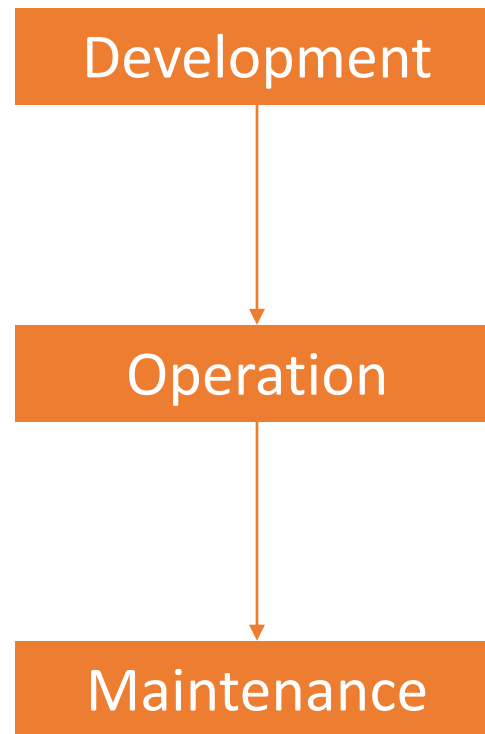
- Non functional properties
 - Are difficult to engineer and to verify
 - Are often forgotten
 - Make the difference between competing products

Software product

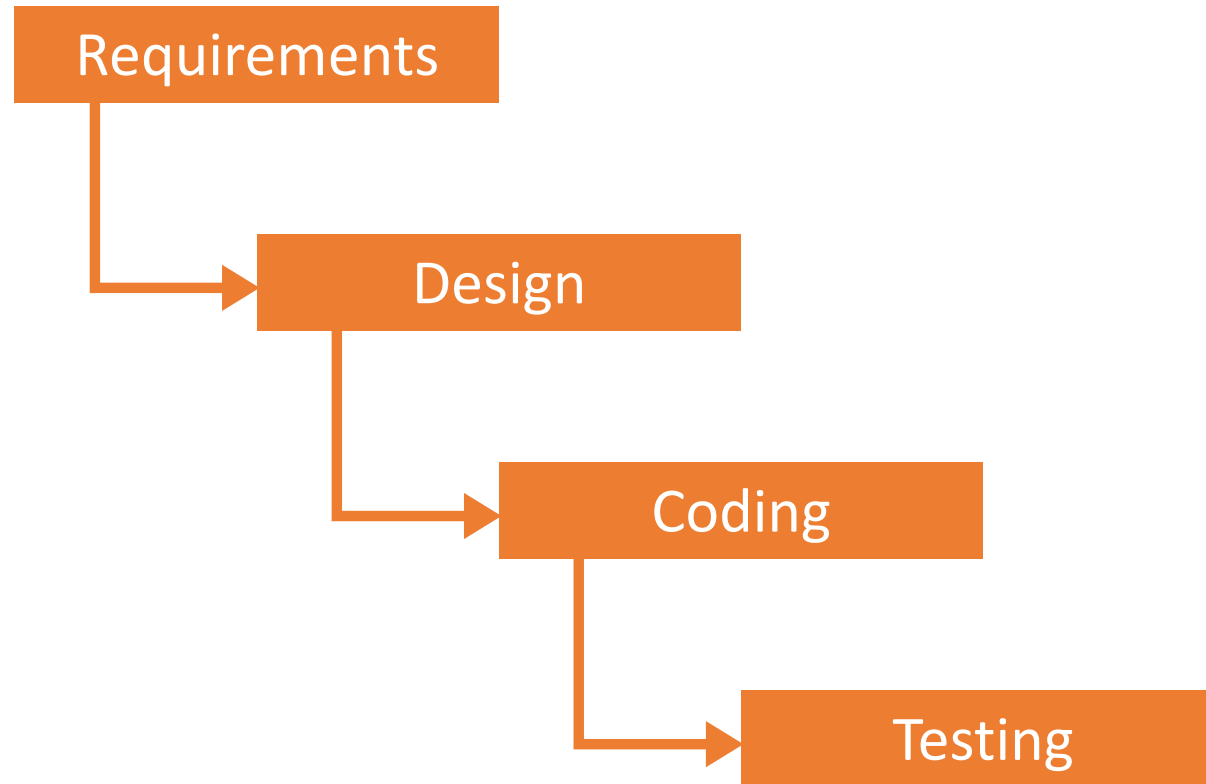
- Functional and non functional properties should be
 - Clearly defined before starting the development
 - Engineered in the product
 - Verified during and after development, before deployment



Software process



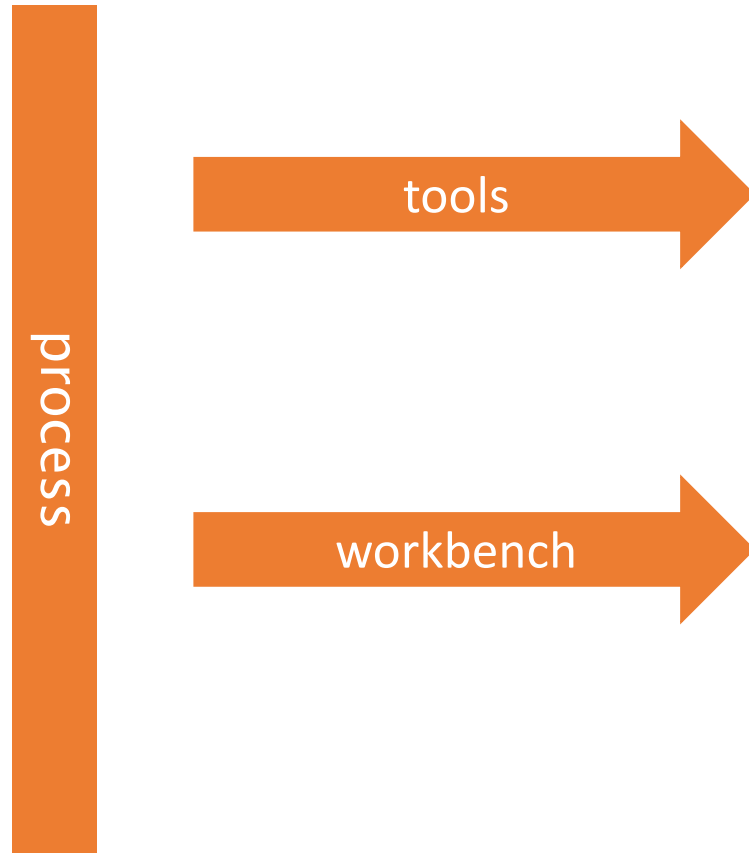
Software process – development phase



Process Properties

- Cost
 - Currency (€, \$, ...)
- Effort
 - Person hours
- Punctuality
 - Promised delivery date vs actual delivery date
- Conformance (to standards, norms)

Classical Workbench



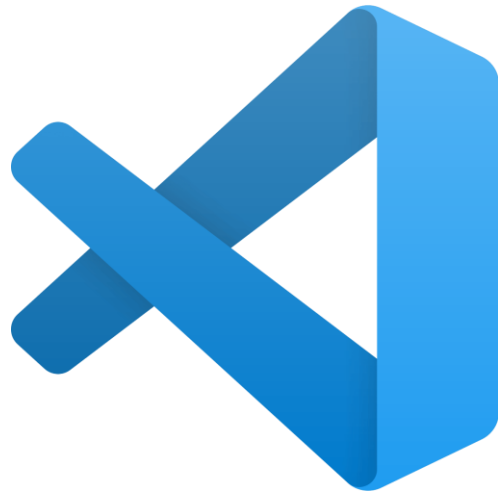
Software Workbench

- Repository + version and configuration management tools
 - Git
 - Subversion
 - Polarion
 - Clearcase
 - Team Foundation Server

Software Tools

- Requirements
 - Context diagram
 - Stakeholders
 - Stories / Personas
 - Use cases
 - Scenarios
 - Functional requirements
 - Non functional requirements
 - Glossary
 -

Software Tools in this course



Software Tools

- Design
 - Component diagram
 - Package diagram
 - Class diagram
 - Interaction diagram
 -

Software Tools

- Development
 - Visual Studio Code
 - IntelliJ-Idea
 - Eclipse
 - Android Studio
 - Xcode
 - PyCharm
 -

Software Tools

- Test
 - Unit test, white box
 - Unit test, black box
 -

Laws

Software Engineering Laws

[Endres Rombach 2005]

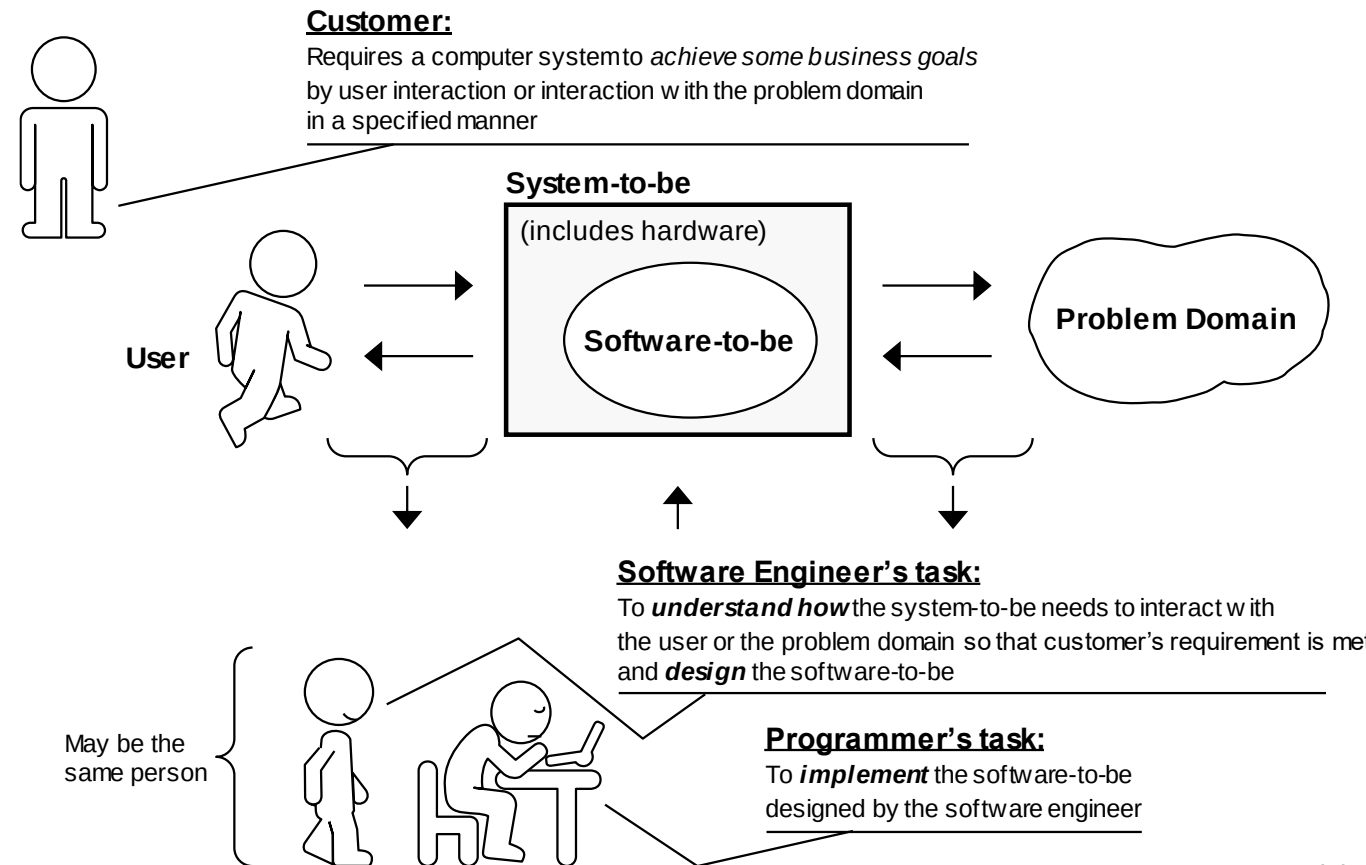
- Requirements deficiencies are the prime source of project failures
- Requirements and design cause the majority of defects
- Defects from requirements and design are the more expensive to fix

Software Engineering Laws

- Modularity, hierarchical structures allow to manage complexity
- Reuse guarantees higher quality and lower cost

Software engineering Laws

- Good designs require deep application domain knowledge



Software Engineering Laws

- Testing can show the presence of defects, not their absence
- A developer is unsuited to test his/her code

Software Engineering Laws

- A system that is used will be changed
- An evolving system will increase its complexity, unless work is done to reduce it
 - Architecture erosion
 - Requirements creep
 - Refactoring

Software Engineering Laws

- Developer productivity varies considerably
- Development effort is a (more than linear) function of size
- Adding resources to a late project makes it later

Software Engineering Laws

- The process should be adapted to the project

Information System laws

- Conway's law
- Structure of a system produced by an organization mirrors the communication structure of the organization
 - Applied to organizational structure: the structure should mirror how members work together
 - Applied to information systems: IS parts will reflect the organizational structure
 - Applied to software projects: module interfaces will reflect the teams' structure

Principles

KISS



Never add
accidental complexity
to
essential complexity

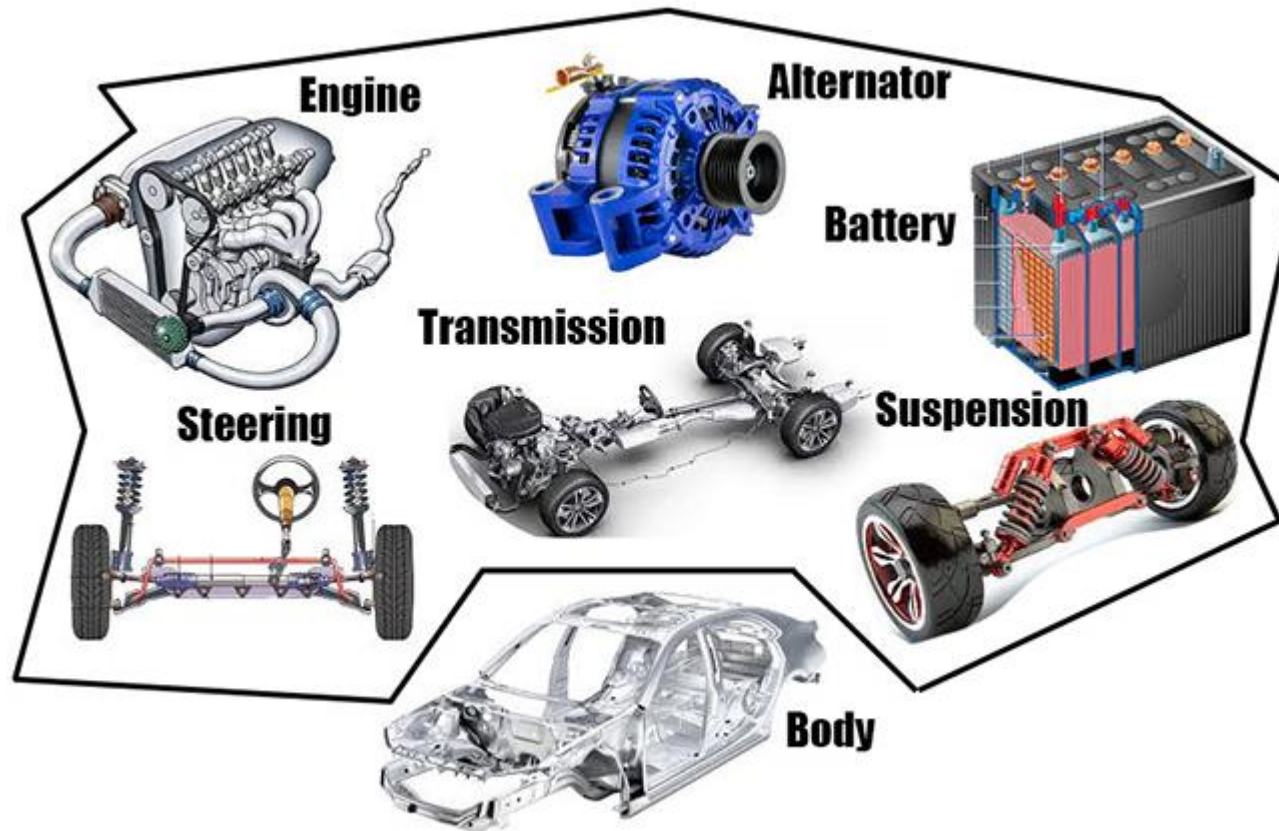
Accidental complexity



Separation of concerns

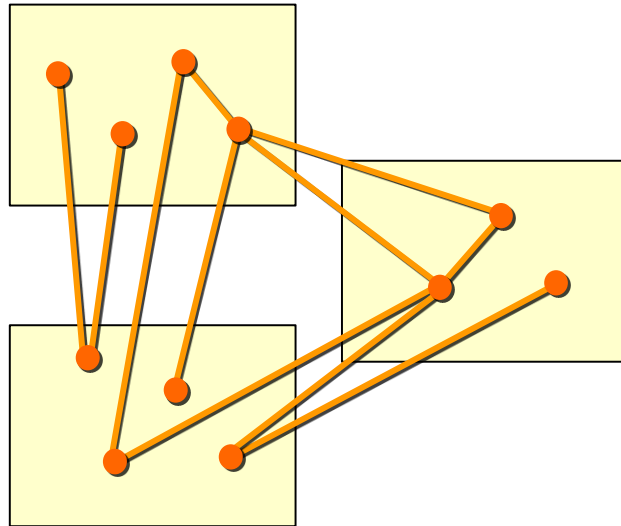
- Given a large, difficult problem, try to split it in many (independent) parts (concerns) , and consider a part at a time:
 - In war: divide and conquer

Separation of concerns (car)

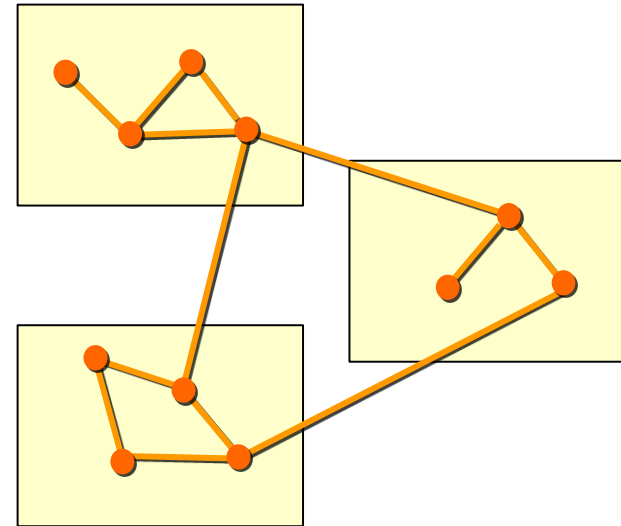


Separation of concerns

- Divide a complex system in modules, with high cohesion and low coupling



high coupling



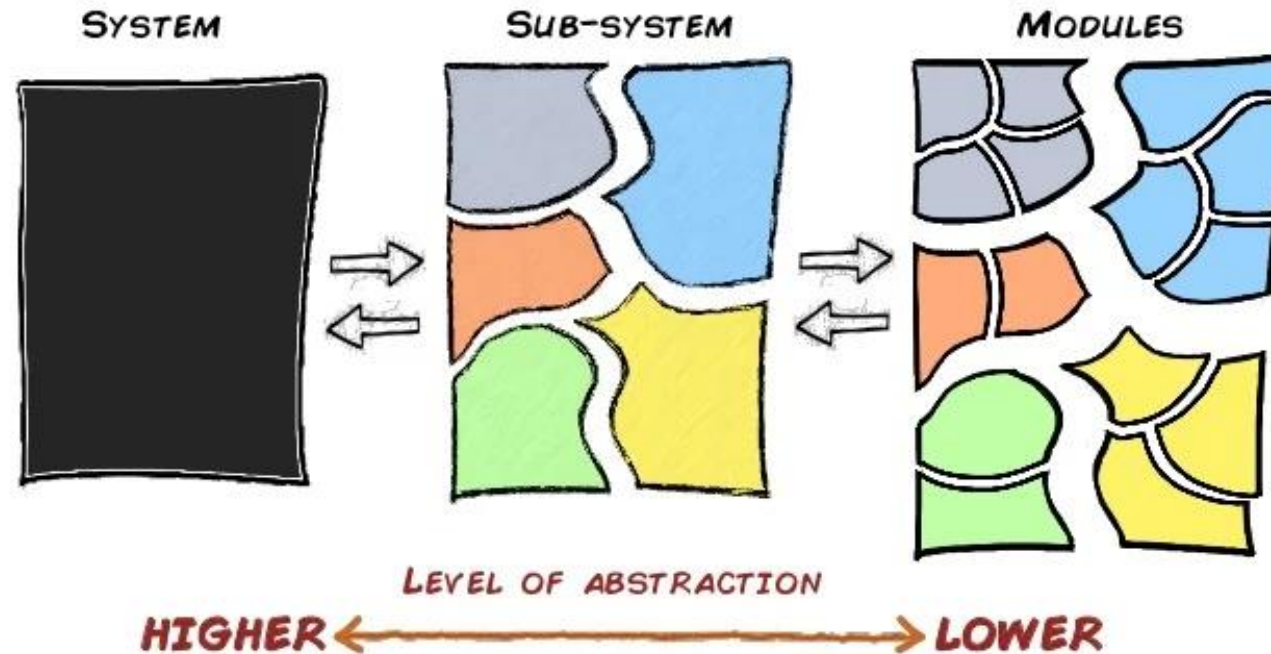
low coupling

Separation of concerns in SE

- Software process
 - Concentrate on what the system should do, then on how, then do it
- Design
 - Split code in (independent) components
- Programming languages
 - Separate error handling and error generation
 - Separate data and function definition

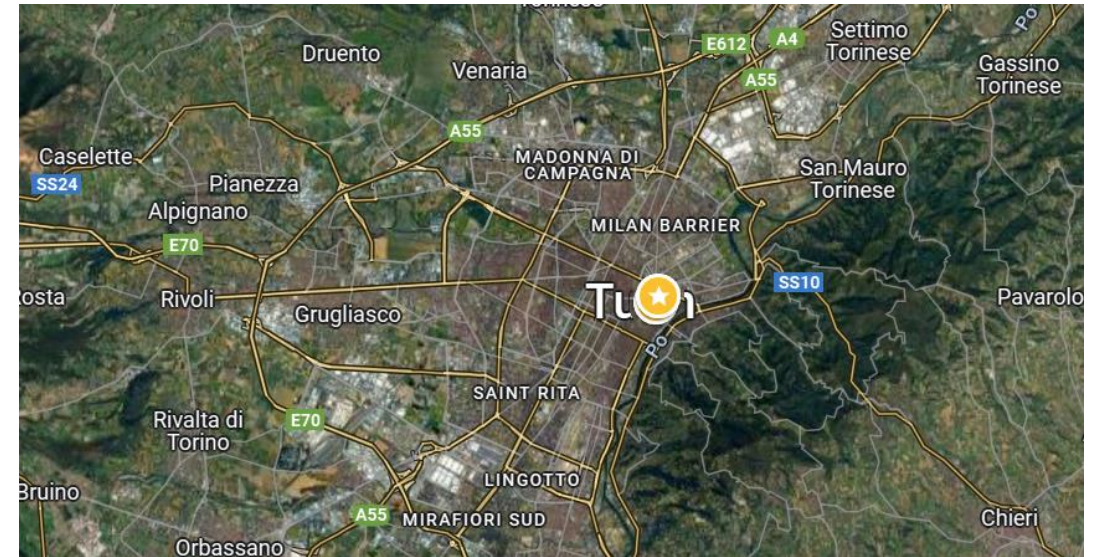
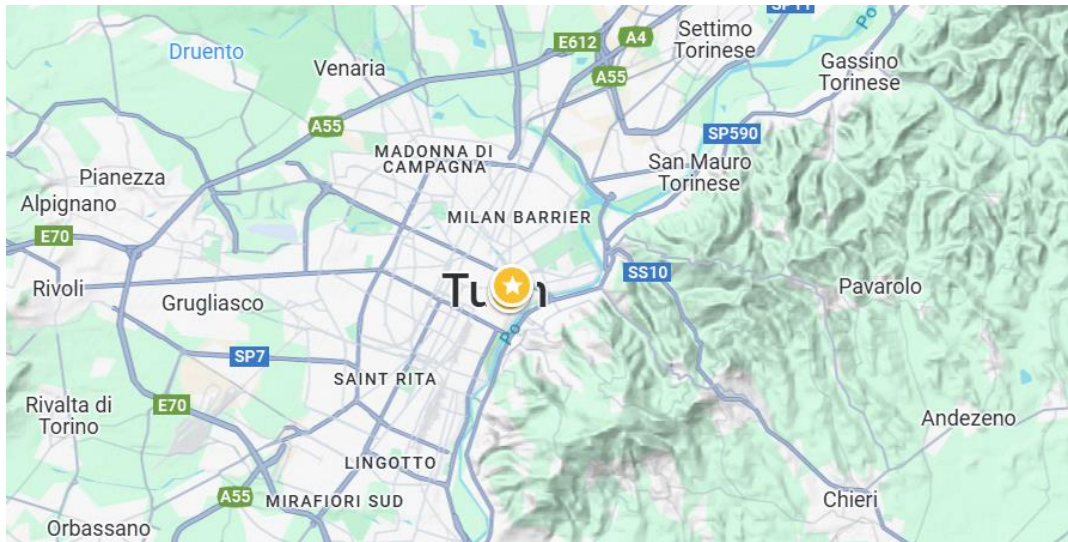
Abstraction

- Given a difficult problem or system, extract a simpler view of it, avoiding unneeded details, then reason on the abstract view (model)



marcello.thiry@gmail.com

Abstraction



Recap

- Software is not only code
- Software process, software products
- Properties
- Laws of Software Engineering and Information Systems
- Principles to be followed